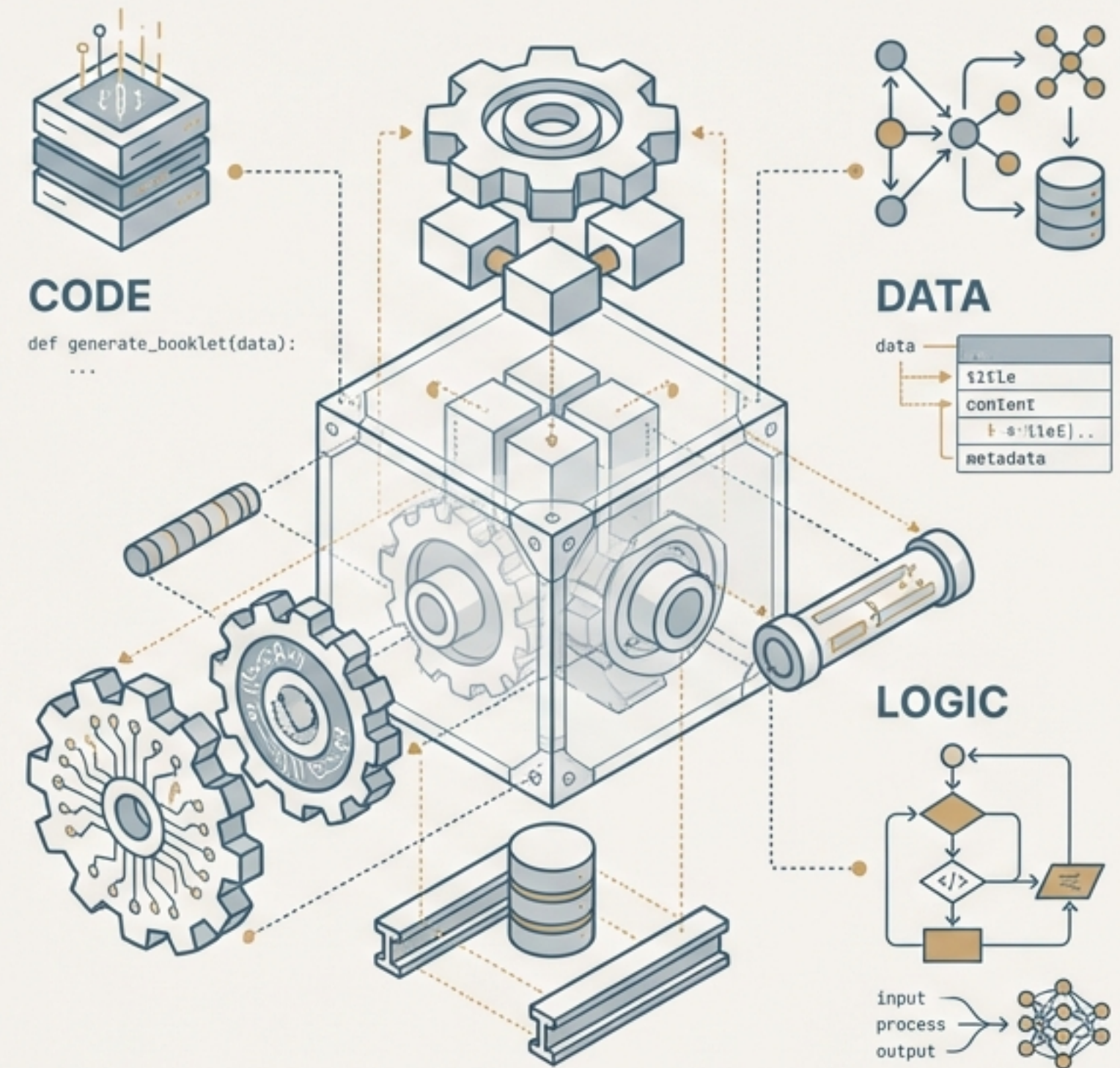


De la Práctica a la Teoría: Creando un Folleto con IA

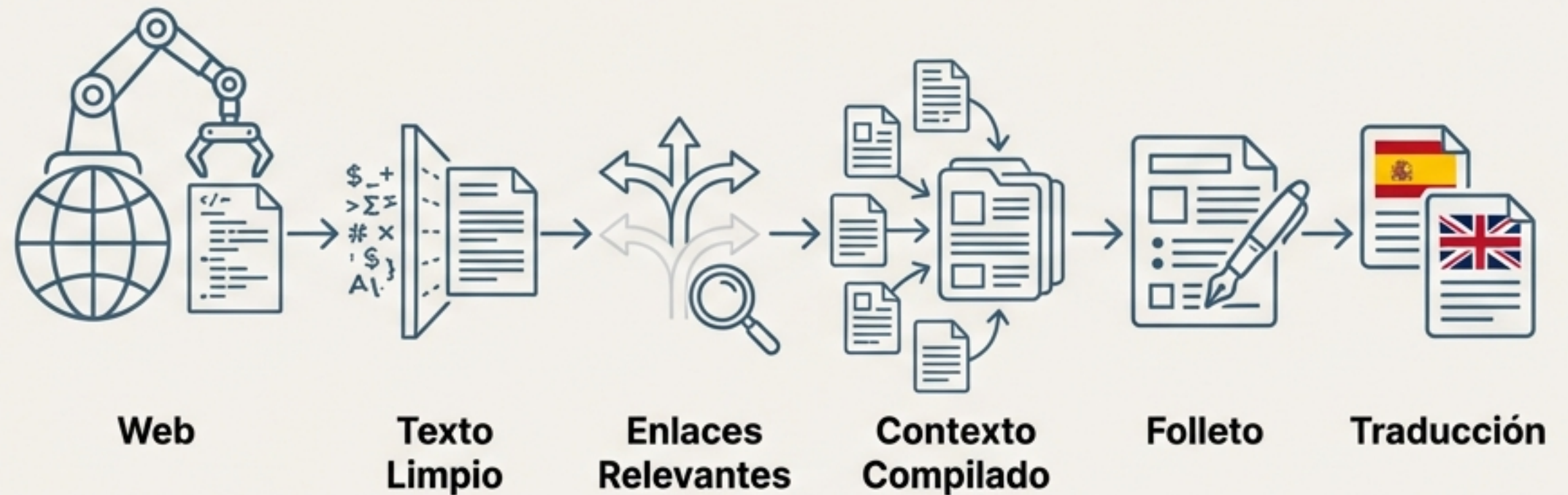
- Tu misión: ir más allá del código.
- Vamos a realizar “ingeniería inversa” a un proyecto práctico.
- Descubriremos los conceptos teóricos detrás de cada línea de código.
- El objetivo final: entender el “porqué” de cada paso que has dado.



Analogía: Desmontar un motor para entender la combustión.
Pasamos de ser mecánicos a ser ingenieros.

El Objetivo: Un Creador de Folletos Inteligente

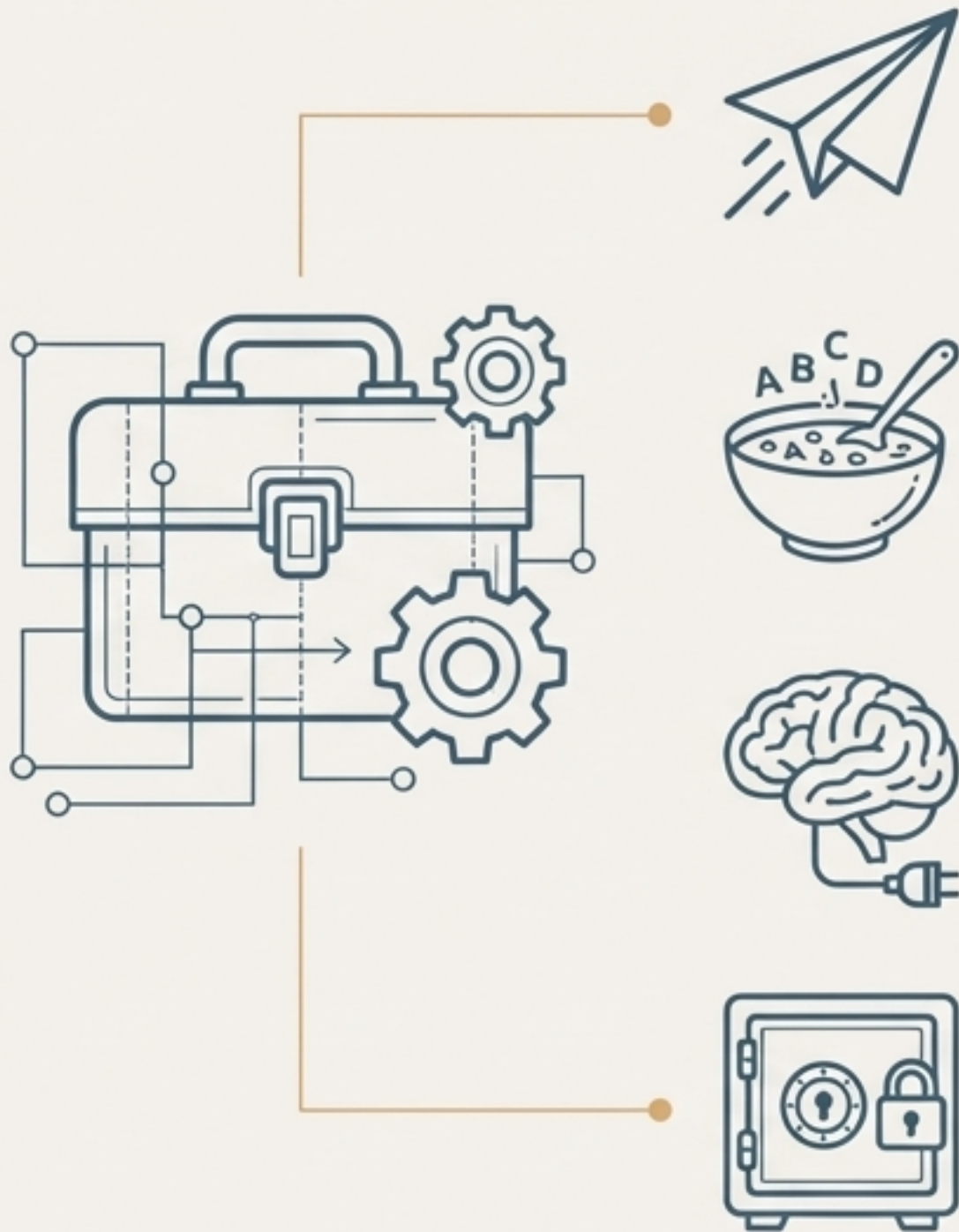
- * **Problema:** Crear un resumen profesional de una empresa es un trabajo manual y lento.
- * **Solución:** Un programa que visita una web, entiende su contenido y redacta un folleto.
- * **Tecnología Clave:** Web Scraping + Modelos de Lenguaje Grandes (LLMs).
- * **Resultado Final:** Un pipeline automatizado que produce un folleto en Markdown y lo traduce.



Concepto Teórico: Pipeline de Datos

Una secuencia de pasos de procesamiento donde la salida de un paso es la entrada del siguiente. Como una cadena de montaje para la información.

La Caja de Herramientas: Librerías y Configuración



``requests``: Nuestro mensajero para pedir páginas web a los servidores.



``BeautifulSoup``: El intérprete que nos ayuda a leer y entender el HTML caótico.



``openai``: El conector directo a la inteligencia artificial de OpenAI.



``python-dotenv``: El guardián de nuestros secretos (la API Key).

Conceptos Teóricos

Librería (Library): Código preescrito por otros que podemos importar para no reinventar la rueda (p. ej. peticiones HTTP).

Clave de API (API Key): Una contraseña única que nos identifica ante un servicio (como OpenAI) y nos autoriza a usarlo.

• **Variables de Entorno (.env):** Un estándar para almacenar configuración sensible (como OPENAI_API_KEY) fuera del código fuente, por seguridad y flexibilidad.

Fase 1: Extrayendo la Materia Prima (Web Scraping)

- * La clase `Website` centraliza toda la lógica de scraping.
- * `requests.get(url)` descarga el código fuente de una web.
- * `BeautifulSoup(html, 'html.parser')` convierte el texto HTML en un objeto navegable.
- * `tag.decompose()` elimina "ruido" (scripts, estilos) que confundiría a la IA.



```
for tag in soup.body(["script", "style", "img"]):  
    tag.decompose()
```

Concepto Teórico: Web Scraping

Técnica de extracción automatizada de información de sitios web.

Concepto Teórico: DOM (Document Object Model)

BeautifulSoup crea una representación en árbol del HTML, permitiéndonos buscar y manipular elementos.

Concepto Teórico: Limpieza de Datos (Data Cleaning)

Proceso de eliminar datos irrelevantes o corruptos. Es fundamental para la calidad de la entrada de la IA.

Analogía: El Bibliotecario Digital

`requests` trae el libro (la página web).
`BeautifulSoup` lo abre y extrae solo los capítulos principales, ignorando el ruido.

El Arte de Preguntar: Fundamentos del Prompting

Concepto Teórico: Prompt Engineering

La disciplina de diseñar y optimizar las entradas (prompts) para que un modelo de IA genere las salidas más precisas y deseadas.
No programamos el modelo, lo guiamos.

Zero-shot



Pedir algo directamente, sin ejemplos. ("Clasifica este texto").
Corresponde a la analogía: "Organiza estos documentos."

One-shot



Dar un solo ejemplo antes de la petición real.
Corresponde a la analogía: "Mira, este va en la carpeta Facturas. Ahora organiza el resto."

Multi-shot (o Few-shot)



Dar varios ejemplos para enseñar el patrón y el formato deseado.
Corresponde a la analogía: "Las facturas van aquí. Los contratos aquí. Los informes aquí. Ahora organiza esto."

Fase 2: El Poder del "Multi-shot Prompting"

Puntos clave:

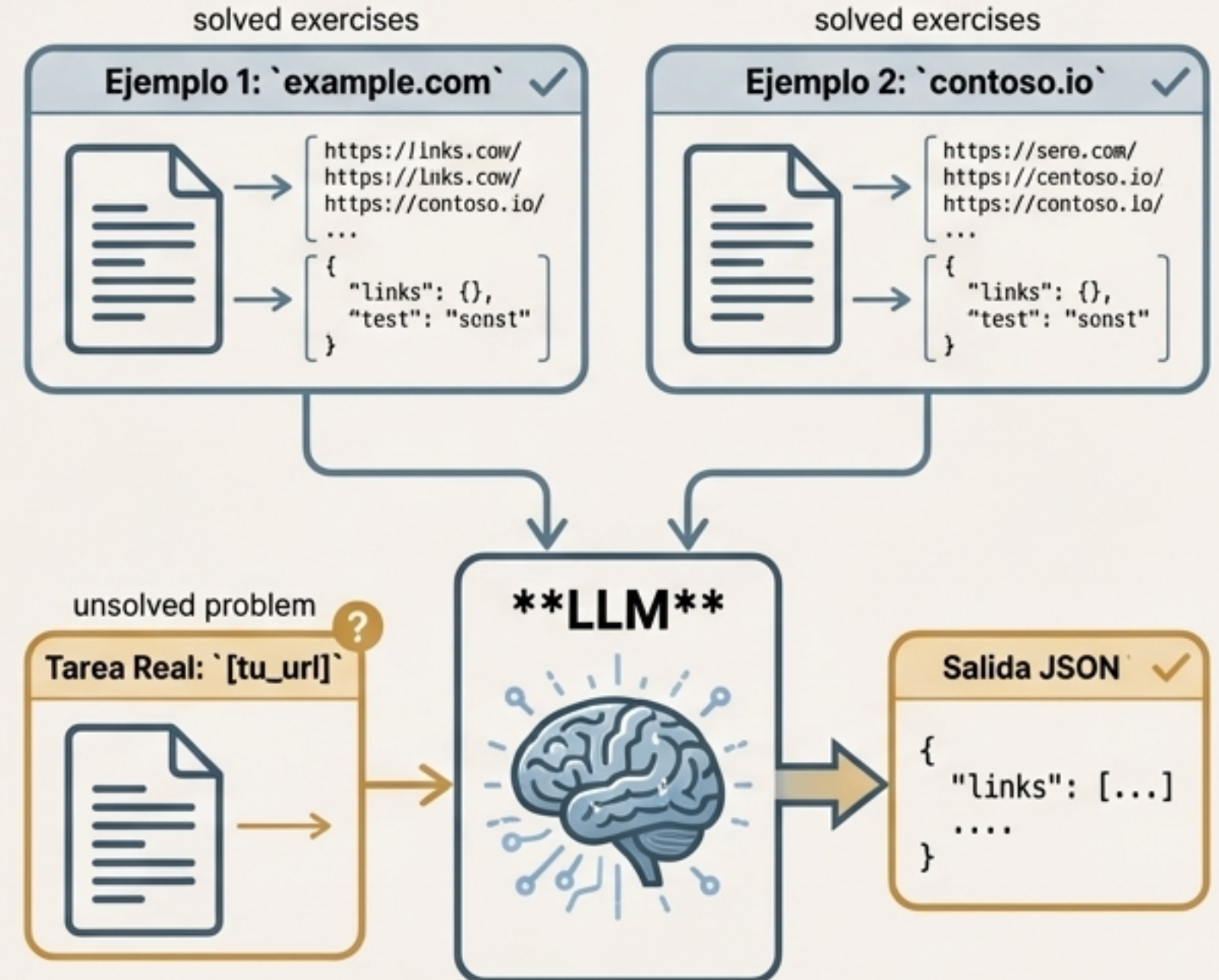
- En el código, la variable `FEWSHOTS` contiene los ejemplos.
- Mostramos a la IA ejemplos de entrada (lista de enlaces) y la salida exacta que queremos (un JSON).
- El modelo aprende el patrón de los ejemplos
- El modelo aprende el patrón de los ejemplos (`example.com`, `contoso.io`).
- Luego, le pasamos nuestra lista real y devuelve el resultado en el formato aprendido.

Concepto Teórico: Aprendizaje en Contexto (In-Context Learning)

La capacidad de los LLMs de aprender una tarea a partir de ejemplos proporcionados en el prompt. Es como darle al modelo ejercicios resueltos antes del examen.

Ejemplo del Código:

```
# La clave: combinar ejemplos con la nueva pregunta
msgs = [*FEWSHOTS, {"role": "user", "content": f"Site: {w.url}..."}]
```



Comunicación Precisa: Forzando la Salida a JSON

Sin JSON

Vivo en la **calle del Sol**,
número **5**, en **Madrid**,
código postal **28013**

Con JSON

```
{  
  "calle": "Calle del Sol",  
  "numero": 5,  
  "ciudad": "Madrid",  
  "cp": "28013"  
}
```

Puntos clave:

- **JSON** (JavaScript Object Notation) es un formato estándar para intercambiar datos.
- Es legible para humanos y fácil de procesar para las máquinas (`json.loads()`).
- Usar `response_format={'type': 'json_object'}` en la llamada a la API es una garantía.
- Esto obliga al modelo a devolver siempre un JSON sintácticamente correcto.

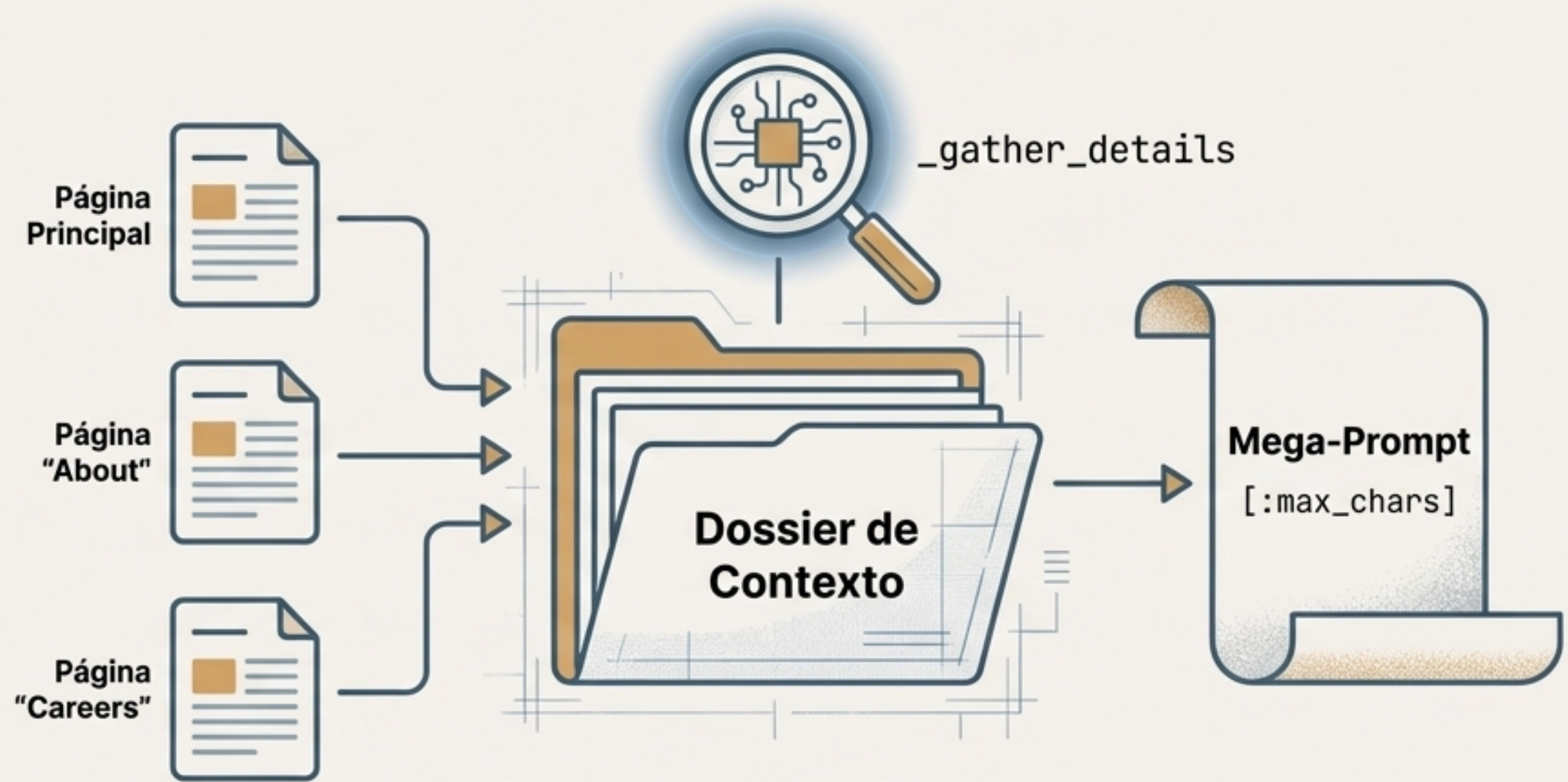
Concepto Teórico:

- **Salida Estructurada (Structured Output):** Una técnica para que los LLMs no generen texto libre, sino datos que se ajustan a un esquema predefinido.
- Hace que la integración de la IA en aplicaciones de software sea robusta y fiable.
- Es el puente entre el lenguaje natural y el software.

Fase 3: Construyendo el Dossier de Inteligencia

Puntos clave:

- La función `_gather_details` es la encargada de esta fase.
- Primero, obtiene el contenido de la página principal.
- Luego, usa `get_links()` para obtener las URLs relevantes.
- Finalmente, visita cada URL, extrae su texto y lo une todo en un gran documento.



Concepto Teórico: Creación de Contexto (Context Stuffing)

Proceso de recopilar y consolidar toda la información relevante de múltiples fuentes en una única entrada (prompt) para que el LLM tenga una visión completa.

Analogía del Detective:

Antes de resolver un caso, un detective reúne todos los informes, testimonios y pruebas en un solo expediente. La función ``_details`` crea ese "expediente del caso" para la IA.

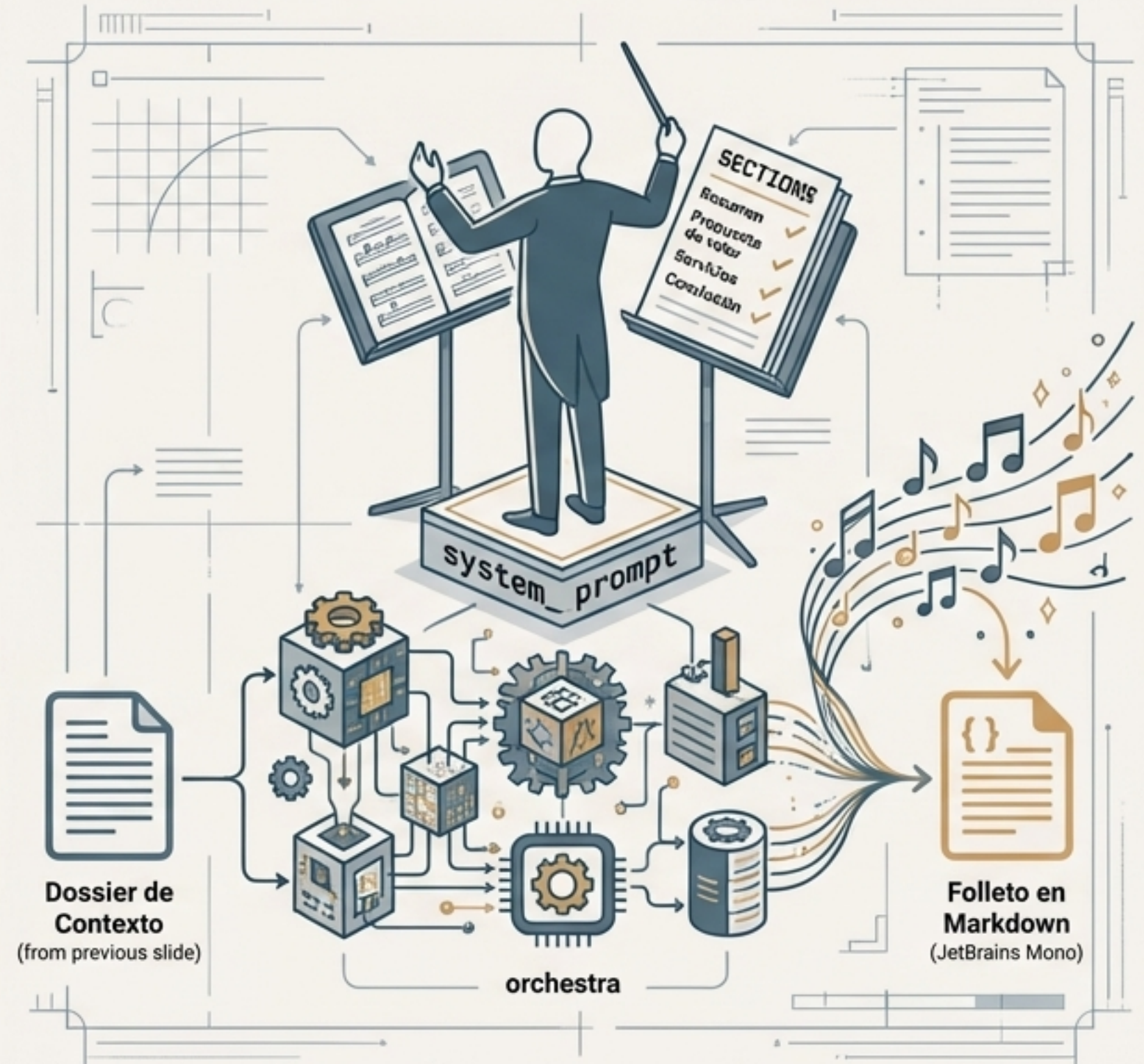
Fase 4: La Gran Final - Redacción del Folleto

Puntos clave:

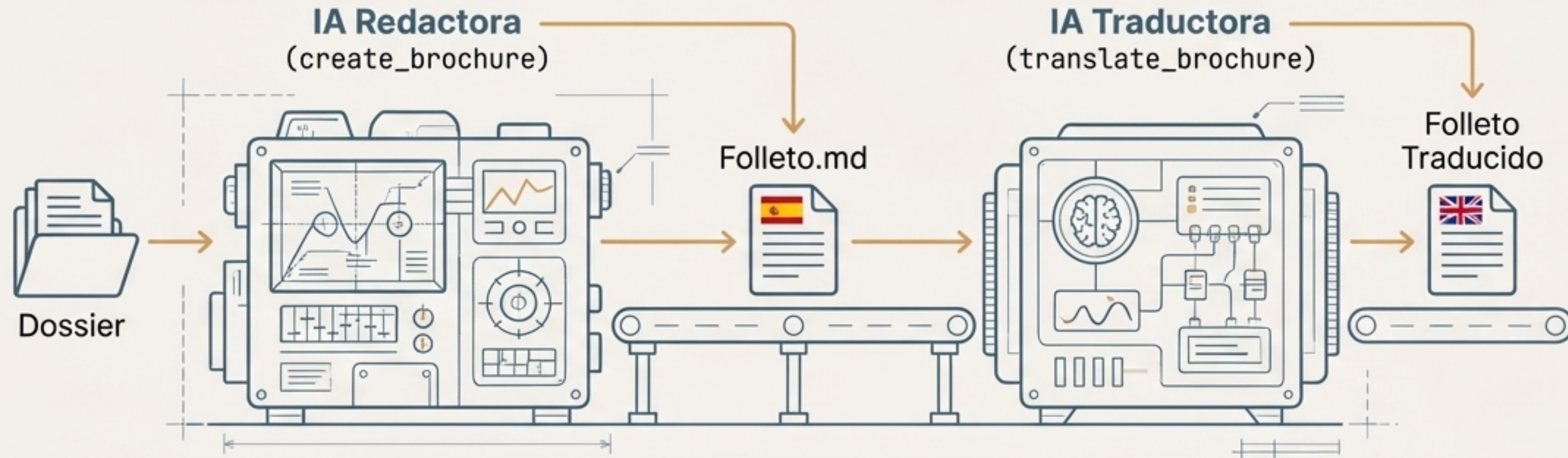
- Se utiliza un `system_prompt` que actúa como el “director de orquesta” (variable `BROCHURE_SYS`).
- Define el rol de la IA: “Eres un asistente que crea un folleto corporativo”.
- Establece reglas claras: usa solo la info suministrada, sigue las secciones de la variable `SECTIONS`, no inventes datos.
- El `user_prompt` le entrega el “dossier” creado en la fase anterior.

Concepto Teórico:

- **Asignación de Rol y Restricciones (Role-playing & Constraints):** Una técnica de prompt engineering donde se le da a la IA una “personalidad” y un conjunto de reglas estrictas. Esto enfoca su creatividad y asegura que la salida cumpla los requisitos.



Fase 5: Un Toque Global - La Traducción



Puntos clave:

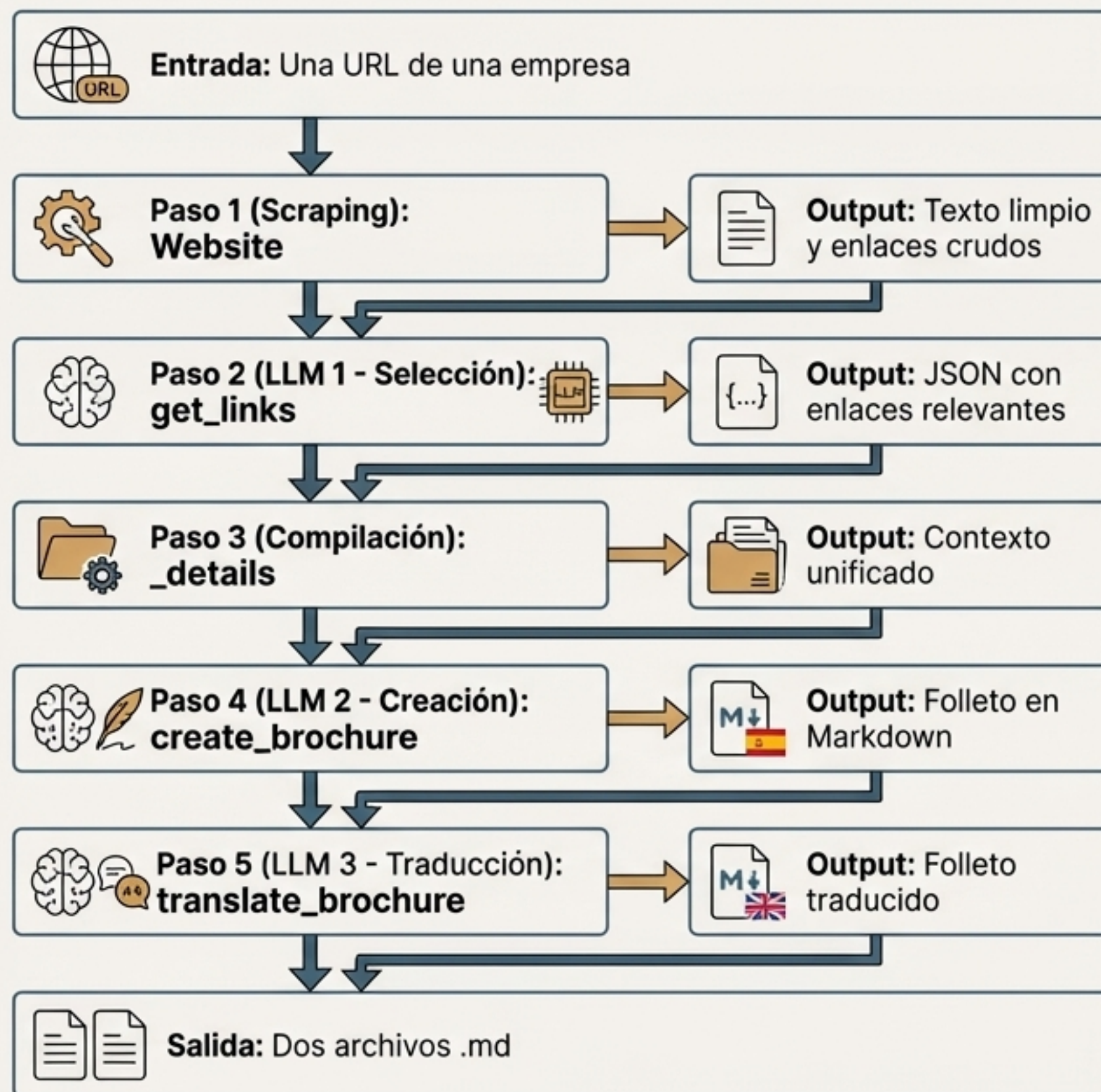
- Una tercera llamada a la API, completamente independiente, en la función `translate_brochure`.
- Utiliza un `system_prompt` muy especializado: "Eres un traductor técnico profesional. Conserva formato Markdown/HTML".
- La entrada es el folleto en Markdown ya generado.
- La salida es el mismo texto, pero en otro idioma (`target_lang`).

Concepto Teórico y Analogía:

****Modularidad y Descomposición de Tareas:****

Es más efectivo descomponer un problema complejo en tareas más pequeñas y especializadas. En una empresa, **no le pides al contable que diseñe el logo**. Aquí hacemos lo mismo: una IA "redactora" y una IA "traductora".

El Pipeline Completo: Visión de Conjunto

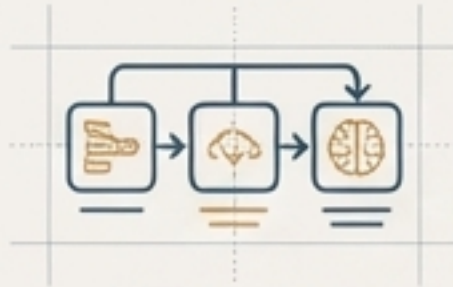


Concepto Teórico: Orquestación de LLMs (LLM Chaining)

Diseño de sistemas donde múltiples llamadas a modelos de IA se encadenan en una secuencia lógica para resolver un problema complejo.

Nuestro proyecto es un ejemplo práctico de encadenamiento de tres llamadas.

Conclusiones: Lo que Hemos Aprendido



Una aplicación de IA robusta es un **pipeline** con varias etapas (Scraping, Selección, Creación).



La **calidad de los datos de entrada** (limpieza de HTML) es fundamental para un buen resultado.



El **Multi-shot Prompting** nos da control y consistencia sobre la IA.



La **salida estructurada (JSON)** es vital para integrar la IA en software de forma fiable.



Descomponer tareas complejas en llamadas a IA más pequeñas y especializadas es una buena práctica de diseño.

Estos principios son universales y aplicables a cualquier proyecto de IA.

Posibles Errores Comunes y Soluciones



`AuthenticationError` (API Key inválida)

Causa: El archivo `.env` no existe, está mal escrito o la clave es incorrecta.

Solución: Verificar el nombre del archivo (`.env`), el nombre de la variable (`OPENAI_API_KEY`) y copiar la clave de nuevo.



Bloqueo de Scraping (HTTP 403 Forbidden)

Causa: El sitio web detecta y bloquea nuestro script.

Solución: Asegurarse de tener un ``User-Agent`` realista en los ``headers``. Probar con otro sitio web.



JSON inválido devuelto por la IA

Causa: Un prompt poco claro o un modelo 'demasiado creativo'.

Solución: Usar ``response_format={'type': 'json_object'}``. Mejorar los ejemplos del multi-shot.

Concepto Teórico: Manejo de Errores (Error Handling)

Una parte crucial de la programación que consiste en anticipar posibles fallos (red, APIs, datos incorrectos) y escribir código para manejarlos (p.ej. bloques ``try...except``) para gestionarlos de forma controlada.